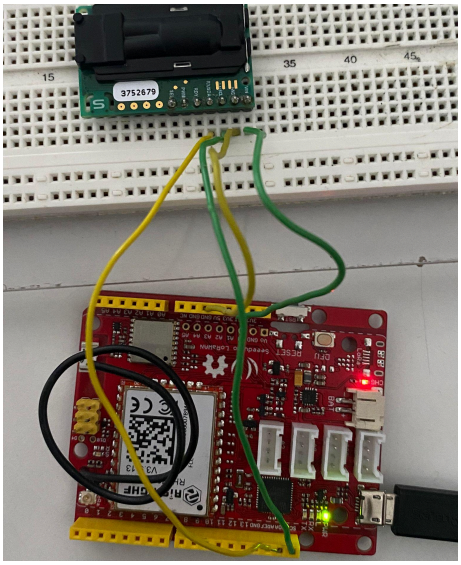
BEKALE Haymard

COMPTE-RENDU SAE 3.01

Carte Lorawan

GND ⇒ branché sur GND (capteur C02)
3v3 ⇒ branché sur VIN (capteur C02) ⇒ alimenté la carte
SCL ⇒ branché sur SCL (capteur C02) ⇒ synchronisé les infos entre la carte et le cap
SDA ⇒ branché sur SDA (capteur C02) ⇒ envoyer/recevoir les informations



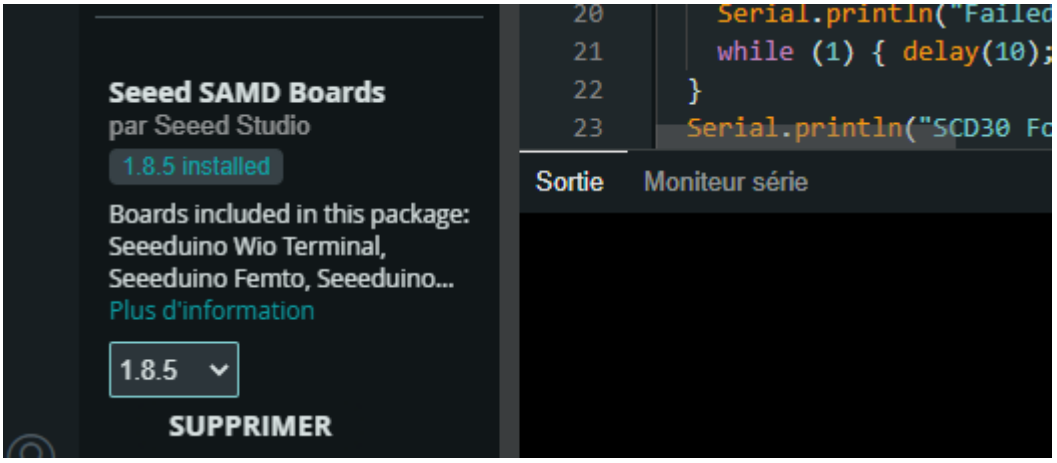
Environnement de travail Arduino

Sur notre IDE arduino pour pouvoir compiler notre programme nous devons installer des paquets un qui gérera la carte et un autre pour les bibliothèques qui seront utilisés.

Pour le gestionnaire de carte nous installons “**seeed SAMD Boards**” mais avant cela dans les préférences nous ajoutons le ce lien

https://files.seeedstudio.com/arduino/package_seeeduino_boards_index.json

Enfin nous cherchons dans le gestionnaire de carte notre board.



Dans le gestionnaire de bibliothèque nous installons ensuite “**Adafruit SCD30**”

Programme

Exemple OTAA que nous avons modifié

OTAA (Over-the-Air Activation)

L'activation en direct (OTAA) est le moyen préféré et le plus sûr de se connecter au réseau The Things Network. Les appareils exécutent une procédure conjointe avec le réseau, au cours de laquelle un DevAddr dynamique est attribué et des clés de sécurité sont négociées avec l'appareil.

```
#include <LoRaWan.h>
#include <Adafruit_SCD30.h>
Adafruit_SCD30 scd30;

void setup(void) {
  Serial.begin(9600);
  while (!Serial)
    delay(10);

// Imprime un message à l'écran de la console indiquant le début du
test du capteur SCD30
  Serial.println("Adafruit SCD30 test!");
// Vérifie si le capteur SCD30 est connecté
  if (!scd30.begin()) {
// Imprime un message d'erreur à l'écran de la console
    Serial.println("Failed to find SCD30 chip");
// Entre dans une boucle infinie pour empêcher toute exécution
ultérieure
    while (1)
      delay(10);
  }
// Imprime un message à l'écran de la console si le capteur SCD30 est
connecté avec succès
  Serial.println("SCD30 Found!");

  Serial.print("Measurement Interval: ");
  Serial.print(scd30.getMeasurementInterval());
  Serial.println(" seconds");

  SerialUSB.begin(9600);
  while (!SerialUSB)
    ;
```

```
  lora.init();

  char buffer[256];
  memset(buffer, 0, 256);
  lora.getVersion(buffer, 256, 1);
  SerialUSB.print(buffer);

  memset(buffer, 0, 256);
  lora.getId(buffer, 256, 1);
  SerialUSB.print(buffer);
  //Ici nous avons ajouté les ID de notre application sur TTN et l'ID
de notre carte seeeduino
  lora.setId(NULL, "70B3D57ED0062EDC", "8CF957200007B94C");
  //sur cette ligne la clé qui permet de sécuriser notre connexion à
TTN
  lora.setKey(NULL, NULL, "02CE4FC075502B45A73CA6A0CEAEBB4A");

  lora.setDeciveMode(LWOTAA);
  lora.setDataRate(DR0, EU868);

  lora.setChannel(0, 868.1);
  lora.setChannel(1, 868.3);
  lora.setChannel(2, 868.5);

  lora.setReceiceWindowFirst(0, 868.1);
  lora.setReceiceWindowSecond(869.5, DR3);

  lora.setDutyCycle(false);
  lora.setJoinDutyCycle(false);

  lora.setPower(14);

  while (!lora.setOTAAJoin(JOIN));
}

void loop(void) {
// Vérifie si de nouvelles données sont disponibles à partir du capteur
SCD30
  if (scd30.dataReady()) {
    Serial.println("Data available!");

    if (!scd30.read()) {
      Serial.println("Error reading sensor data");
      return;
```

```
    }
// Imprime la mesure de la température à l'écran de la console
    Serial.print("Temperature: ");
    Serial.print(scd30.temperature);
    Serial.println(" degrees C");
// Imprime la mesure de l'humidité à l'écran de la console
    Serial.print("Relative Humidity: ");
    Serial.print(scd30.relative_humidity);
    Serial.println(" %");
// Imprime la mesure de CO2 à l'écran de la console
    Serial.print("CO2: ");
    Serial.print(scd30.CO2, 3);
    Serial.println(" ppm");
    Serial.println("");
  } else {
    // Serial.println("No data");
  }

  bool result = false;

  char payload[128];
  // Formatez la charge utile avec le même format que le décodeur
JavaScript
  sprintf(payload, "Temp: %.2f; Hum: %.2f; CO2: %.2f",
scd30.temperature, scd30.relative_humidity, scd30.CO2);

  // Envoyez la charge utile à notre passerelle notre payload
  result = lora.transferPacket(payload, strlen(payload));

  if (result) {
    short length;
    short rssi;

    char buffer[256];
    memset(buffer, 0, 256);
    length = lora.receivePacket(buffer, 256, &rssi);

    if (length) {
      SerialUSB.print("Length is: ");
      SerialUSB.println(length);
      SerialUSB.print("RSSI is: ");
      SerialUSB.println(rssi);
      SerialUSB.print("Data is: ");
      for (unsigned char i = 0; i < length; i++) {
        SerialUSB.print("0x");
```

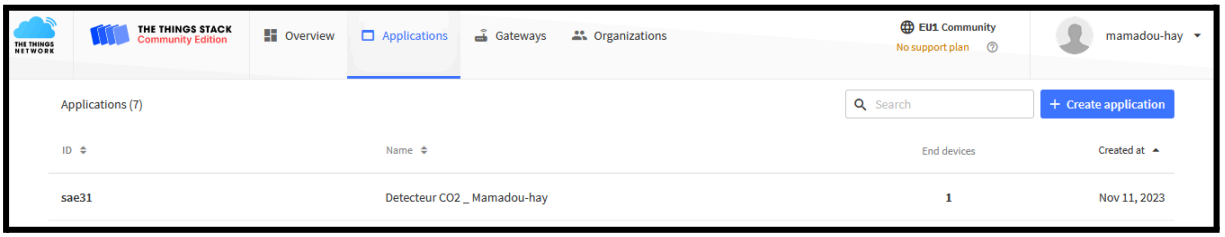
```
        SerialUSB.print(buffer[i], HEX);
        SerialUSB.print(" ");
      }
      SerialUSB.println();
    }
  }

  delay(100);
}
```

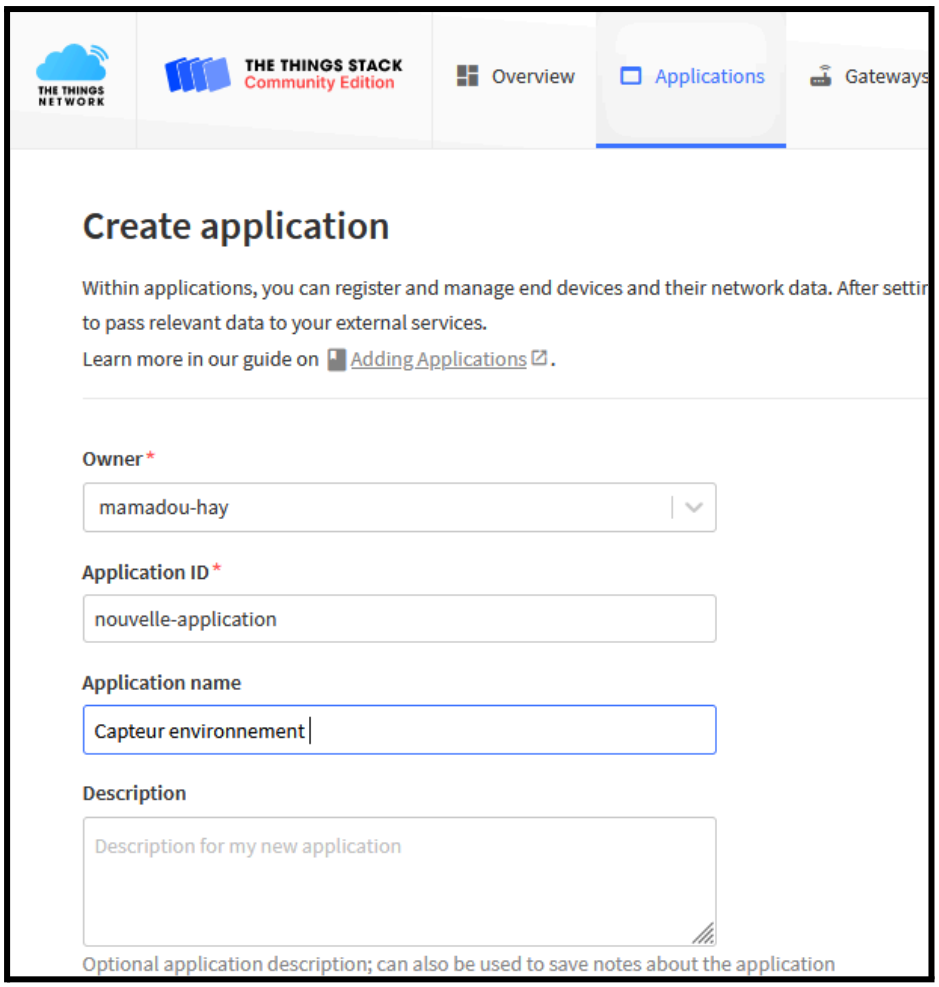
The Thing Network

Application

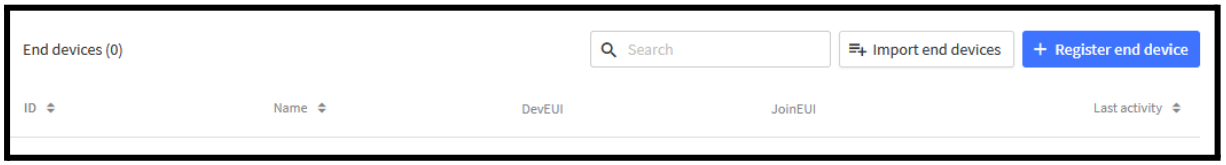
Dans l'onglet application nous créons une nouvelle application.



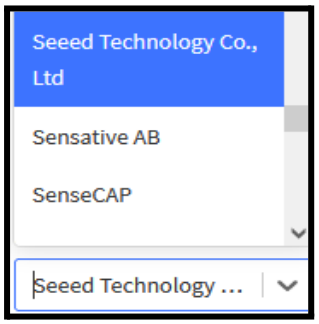
Nous renseignons ensuite les informations nécessaires.



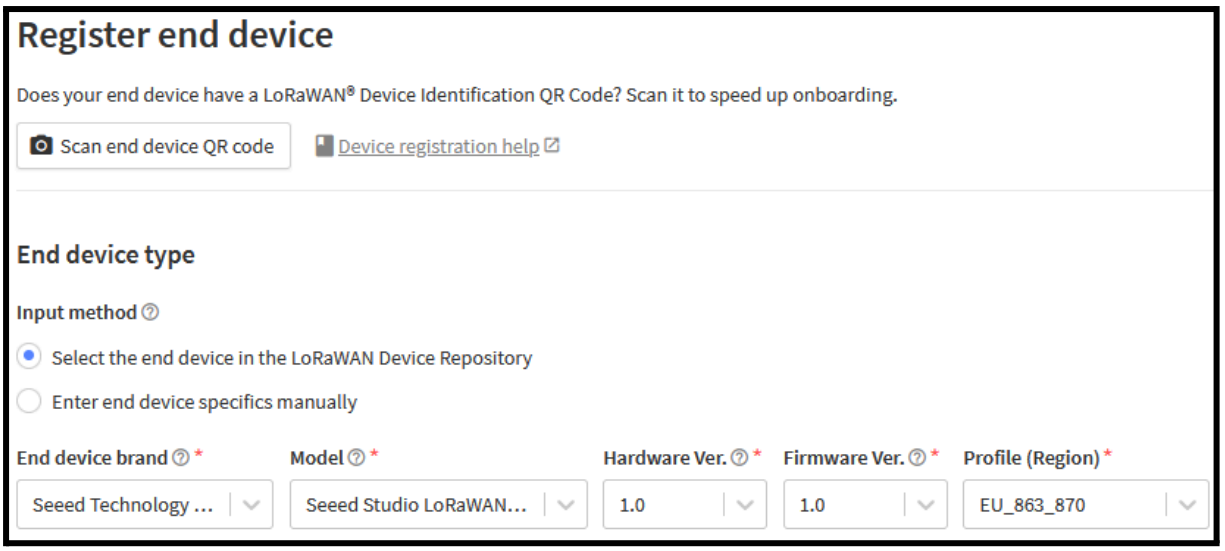
Dans Overview de notre application nous enregistrons les paramètres de notre carte seeeduino.



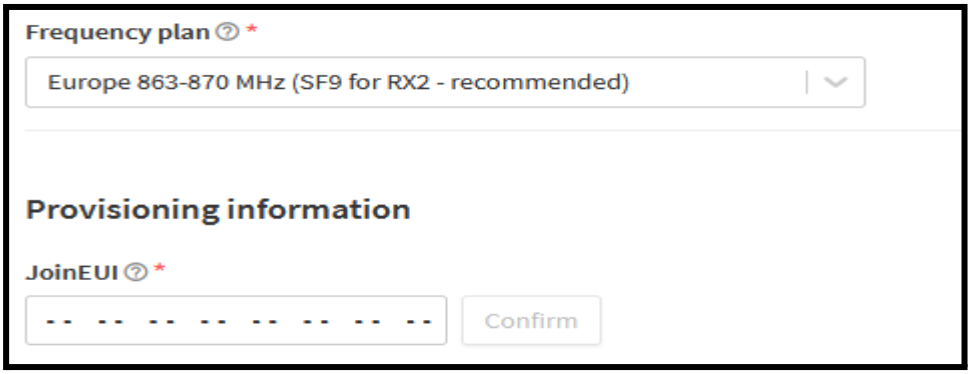
Nous choisissons le type d'appareil donc le type de carte ici c'est "**seeed Technoly Co.Ltd**"



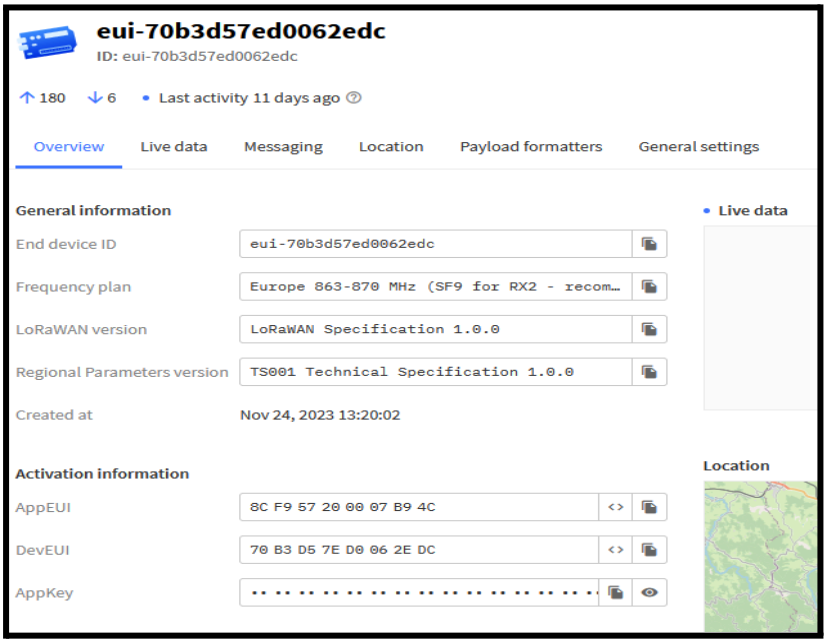
Avec le modèle de carte et le type de fréquence utilisé



enfin nous choisissons la fréquence utilisé recommandé et le **JoinEUI** qui se trouve sur notre microcontrôleur.



Après avoir renseigné toutes ces informations dans End devices nous sommes censé retrouvé ceci.

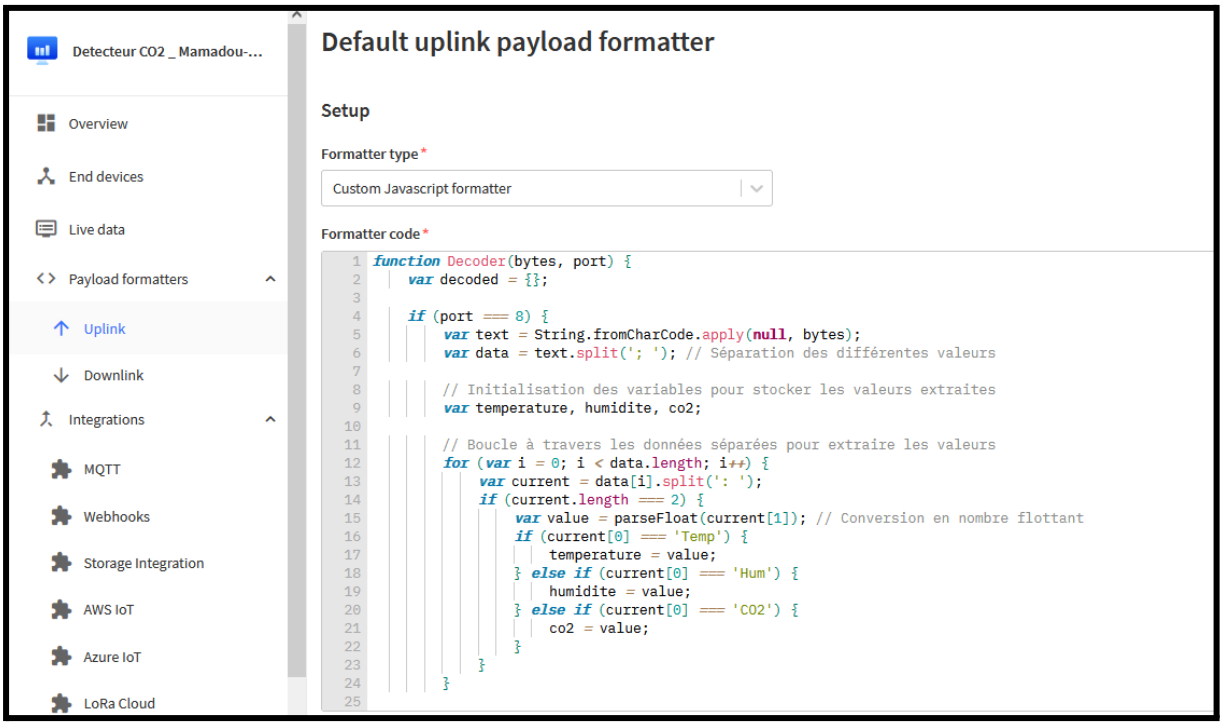


Après toutes ces étapes nous pouvons compiler notre programme sur arduino, sur notre moniteur série nous pouvons observer les informations renvoyé à la passerelle

```
COM3

You can do other work during BME680 measurement
Reading completed at 9772923
Temperature = 29.17 *C
Pressure = 1004.46 hPa
Humidity = 57.64 %
Gas = 66.06 KOhms
```

Sur TTN dans Uplink qui sont des données montantes venant de la passerelle nous devons décoder notre payload en format JSON sinon nous ne récupérerons pas les valeurs exacte de notre charge utile.



Programme Js

```
function Decoder(bytes, port) {
	var decoded = {};

	if (port === 8) {
	var text = String.fromCharCode.apply(null, bytes);
	var data = text.split('; '); // Séparation des différentes valeurs

	// Initialisation des variables pour stocker les valeurs extraites
	var temperature, humidite, co2;

	// Boucle à travers les données séparées pour extraire les valeurs
	for (var i = 0; i < data.length; i++) {
	var current = data[i].split(': ');
	if (current.length === 2) {
	var value = parseFloat(current[1]); // Conversion en nombre flottant
	if (current[0] === 'Temp') {
		temperature = value;
	} else if (current[0] === 'Hum') {
		humidite = value;
	} else if (current[0] === 'CO2') {
		co2 = value;
	}
	}
	}

	return {
	temperature: temperature,
	humidity: humidite,
	CO2: co2
	};
	}

	return decoded;
}
```

Enfin dans live data, nous pouvons visualiser nos données correctement décodé

Intégration

Pour cette SAE nous avons utilisé trois voire quatre méthodes d'intégration différentes avec la méthode **Webhook**, mais pour chacune d'entre elles les versions gratuites sont limitées. Et la méthode **MQTT** avec **Chirpstack**.

- **Ubidots** qui nécessite une très bonne maîtrise du javaScript pour pouvoir récupérer notre payload décoder.

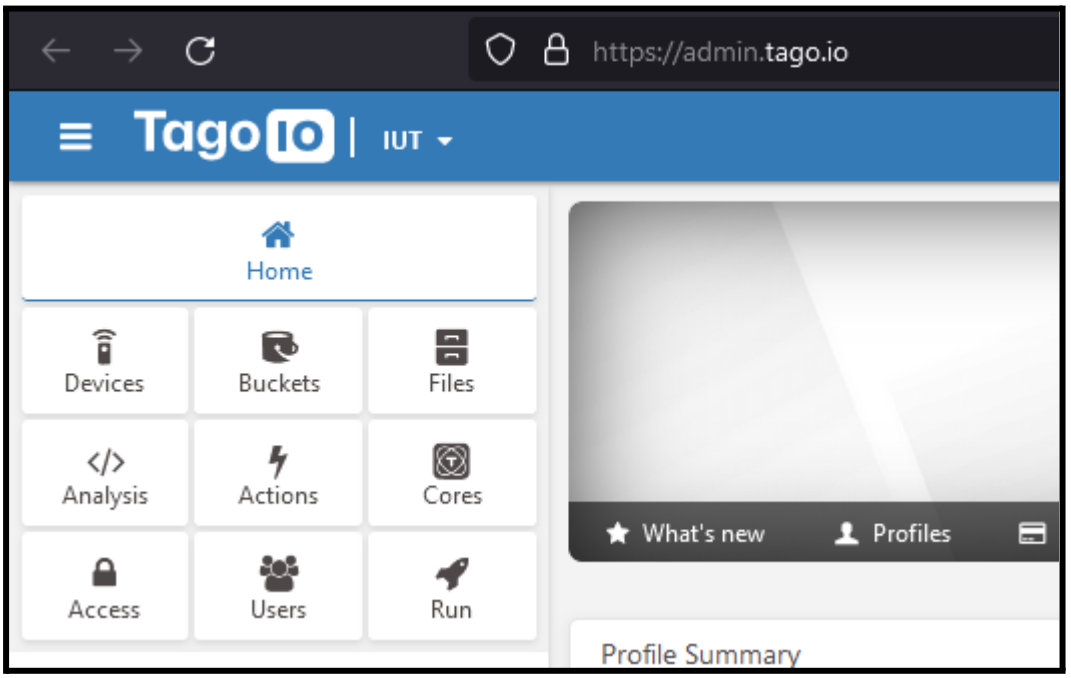
- **The Thingsio** est aussi une très bonne plateforme de visualisation de données sans avoir une trop grosse maîtrise du JavaScript. Mais pour afficher les données du payload sur un graphique pour qu'il soit attractif il faudrait avoir un abonnement.

- **TagoIO** très facile à intégrer à notre application TTN et qui n'est pas très compliqué à configurer pour observer les données de notre charge utile (payload).

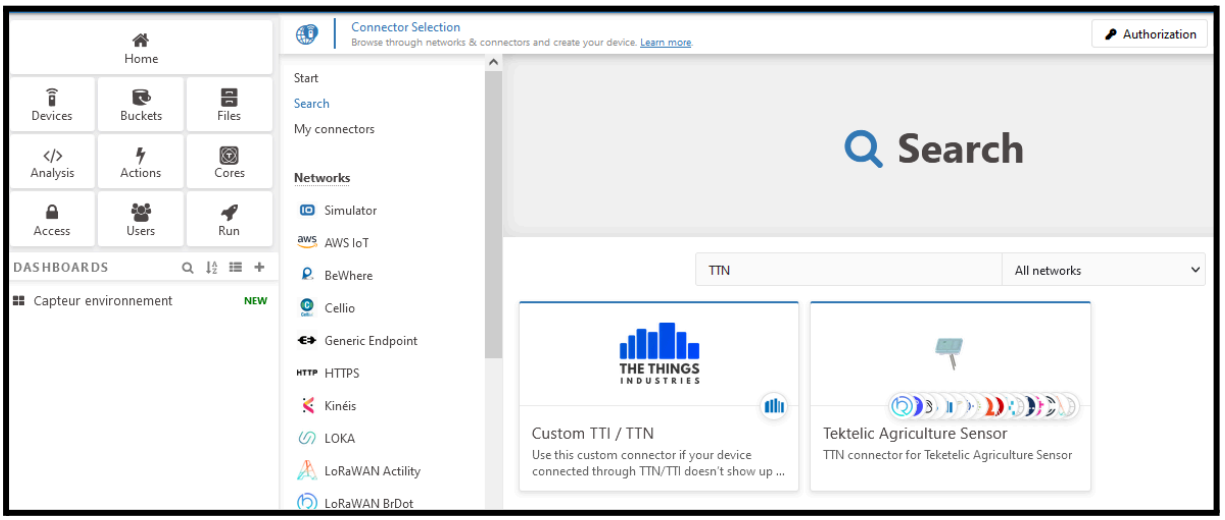
Pour chacune de ces plateformes les intégrations sont différentes et une documentation est disponible sur TTN en utilisant la méthode webhook.

Dashboard - WEBHOOK (TagoIO)

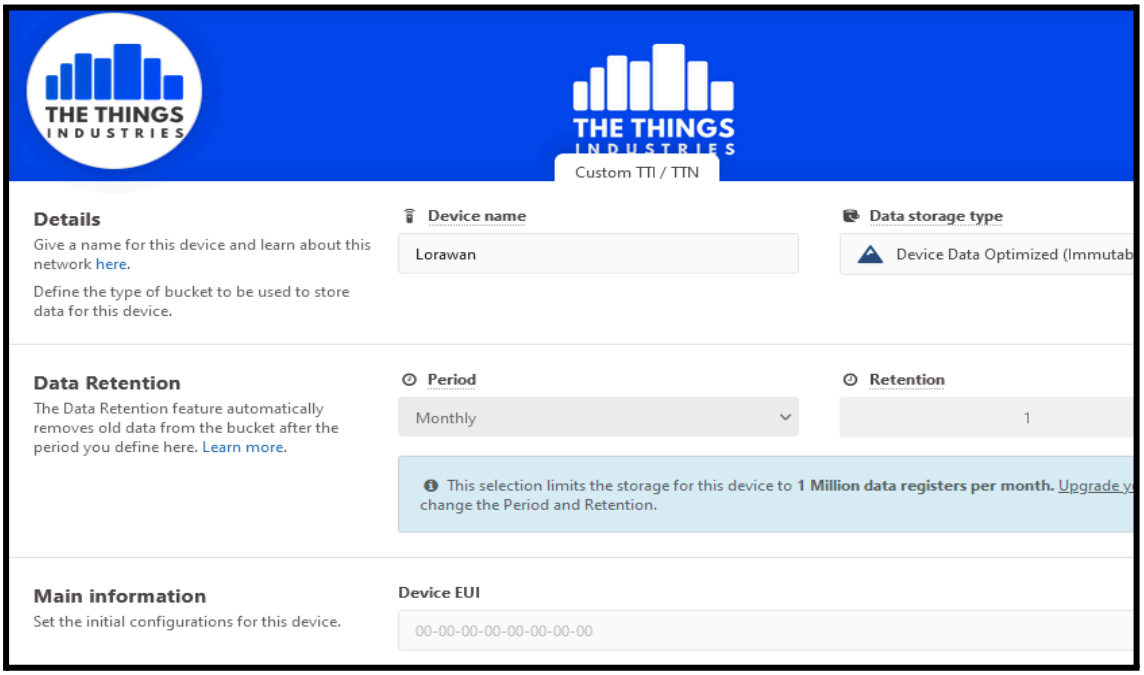
Dans le menu Devices nous créons notre appareil

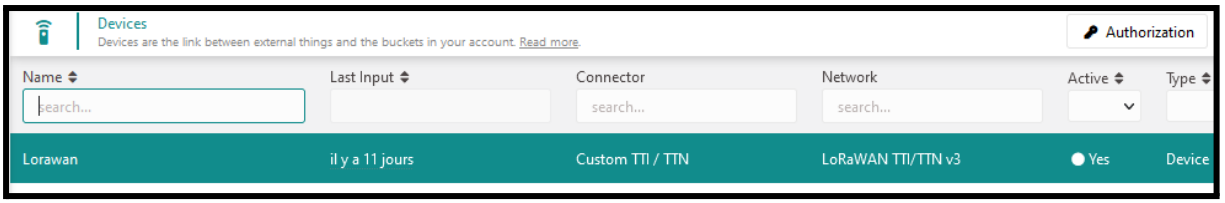


Ensuite nous choisissons la plateforme qui nous renverra les données montantes vers TagoIO dans notre cas c'est TTN.

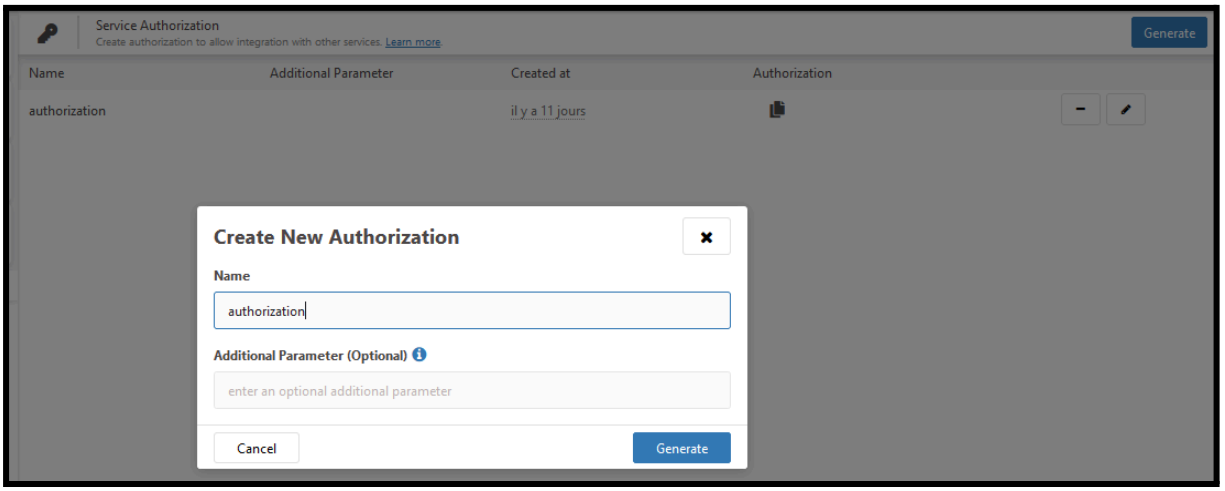


Nous donnons un nom à notre service ensuite nous ajoutons le Dev-EUI de notre application TTN sur notre connecteur TagoIO.

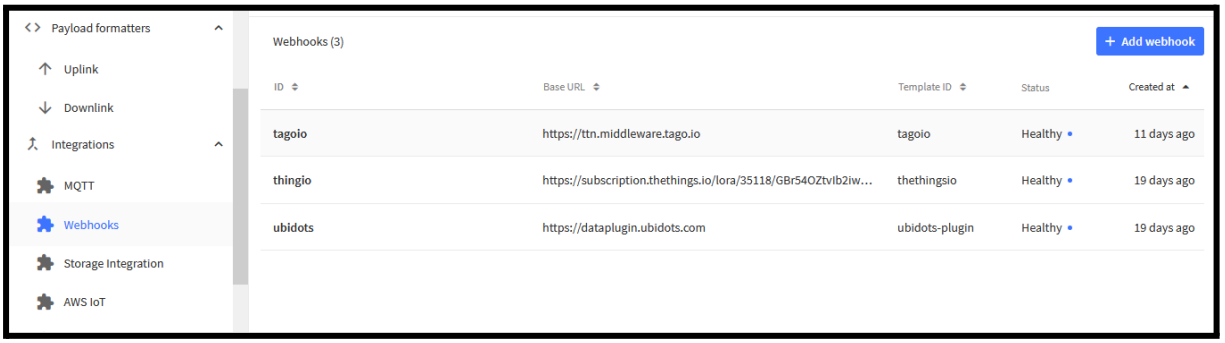




Toujours dans Devices nous générons une autorisation qui permet l'intégration sur TTN.



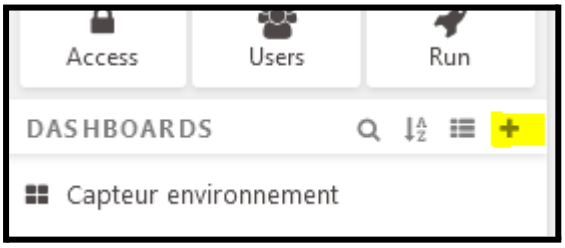
Nous revenons ensuite sur TTN dans l'onglet webhooks nous ajoutons TagoIO.



Nous copions l'autorisation de TagoIO et nous la collons sur celle de TTN.

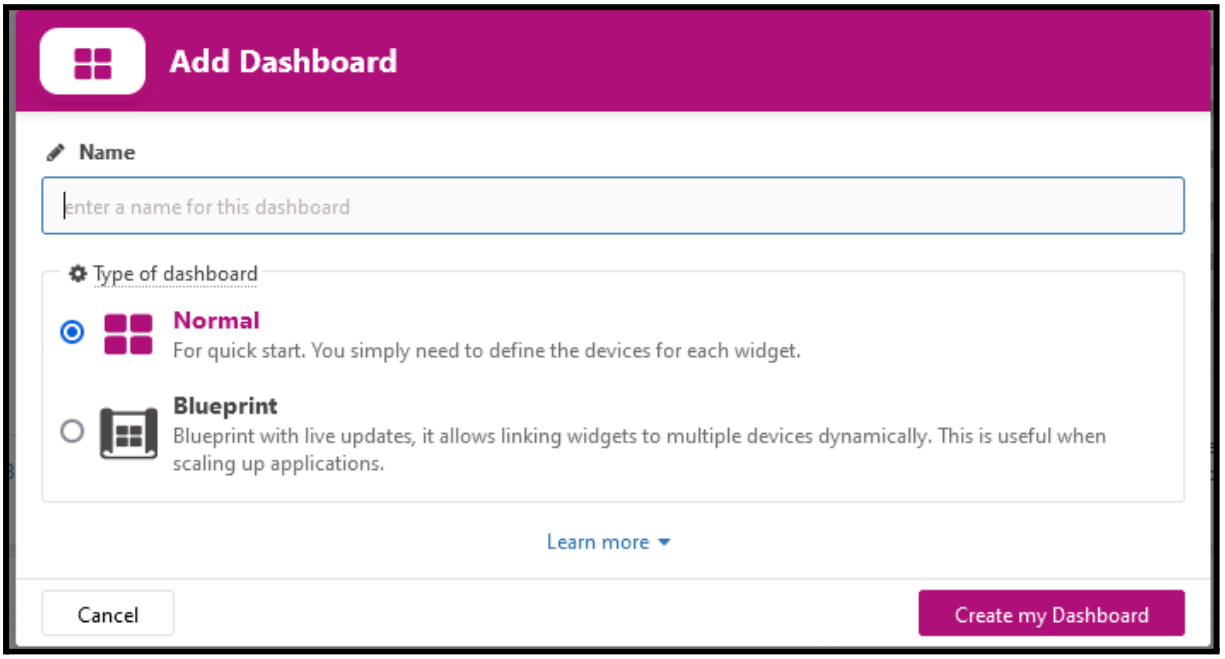
Ce qui conclut l'intégration de TagoIO, lorsque notre intégration est correctement faite nous verrons "**Heathly**" sur notre connecteur TTN.

Nous pouvons enfin passer à la création de notre Dashboard qui est plutôt simple et rapide.



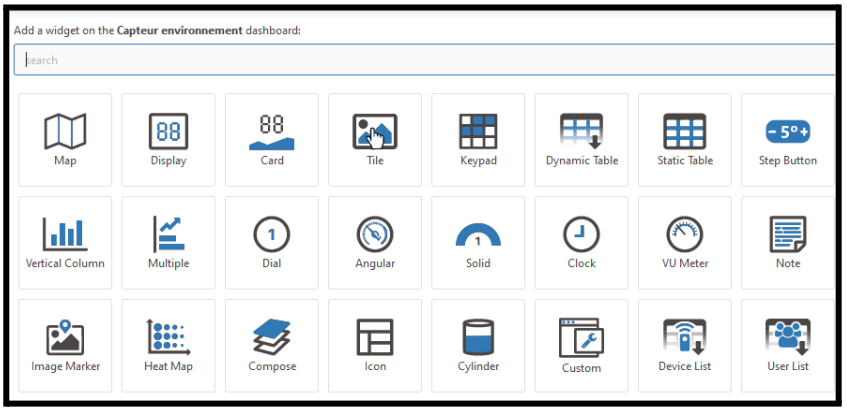
Nous donnons un nom à notre dashboard et le type de dashboard que nous voulons.

- **Normal**: pour un démarrage simple et rapide.
- **Blueprint**: pour l'utilisation de plusieurs widgets et récupérer des metrics de différents systèmes.

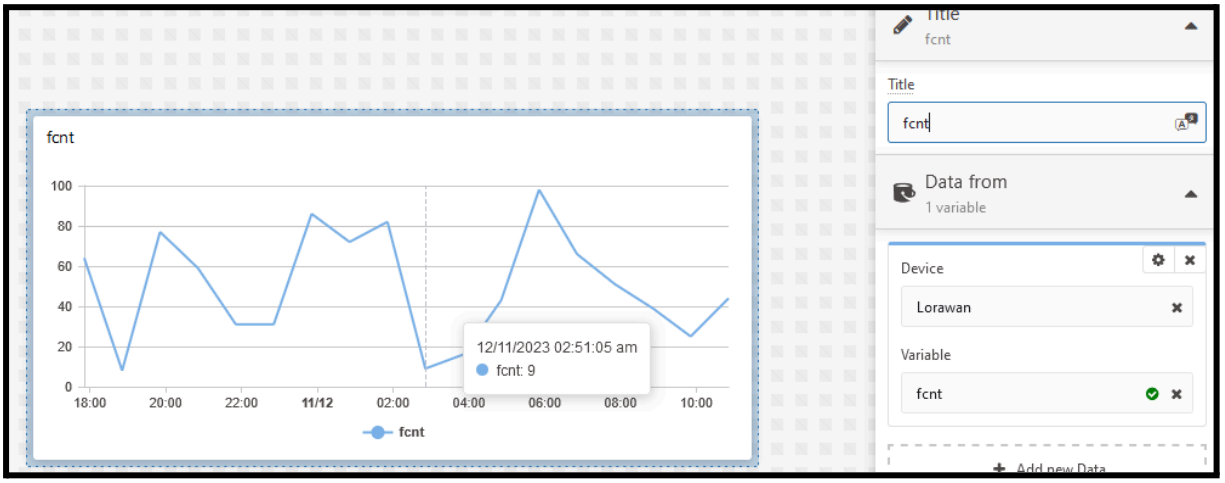


Nous ajoutons un nouveau widget

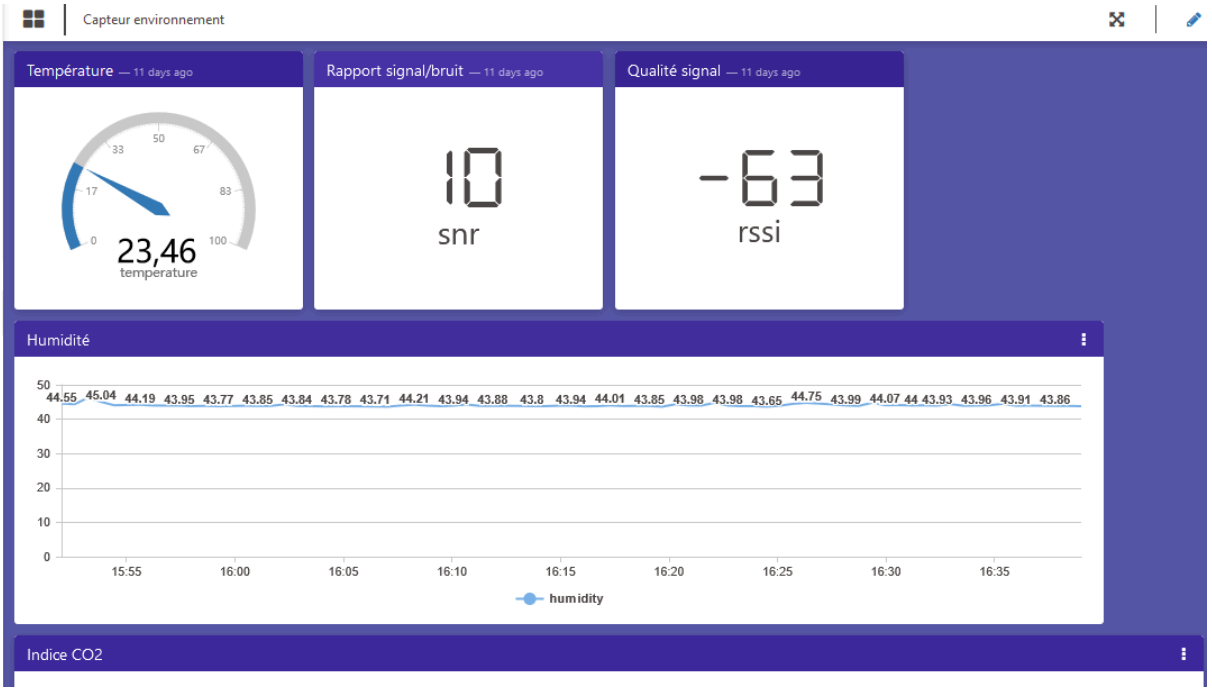
Selon le type de données que nous voulons visualiser nous choisissons notre graphique.



Après avoir choisi notre type de graphe, nous choisissons le service que nous avons créé précédemment et le type de variable que nous voulons afficher. Nous répétons cette opération à chaque fois pour chacune des variable que nous voulons.

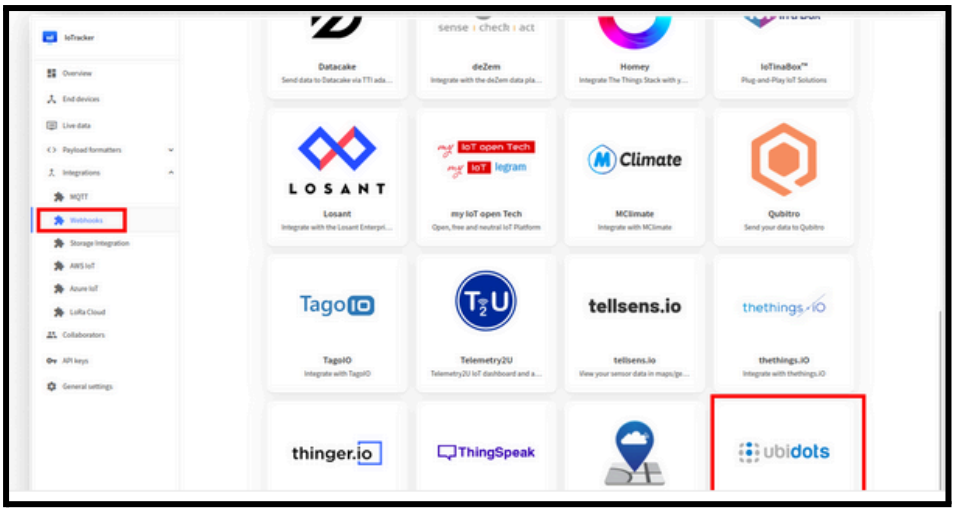


Notre Dashboard avec la température, le SNR, la qualité du signal, l'humidité et le CO2.

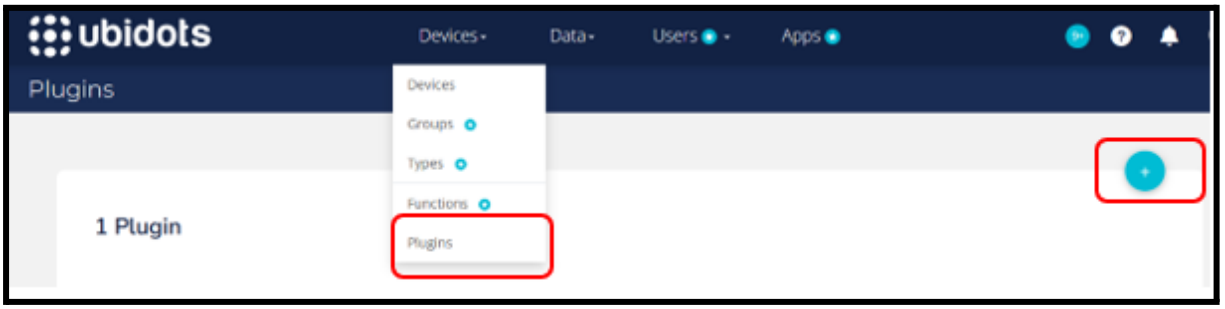


Dashboard - WEBHOOK (Ubidots)

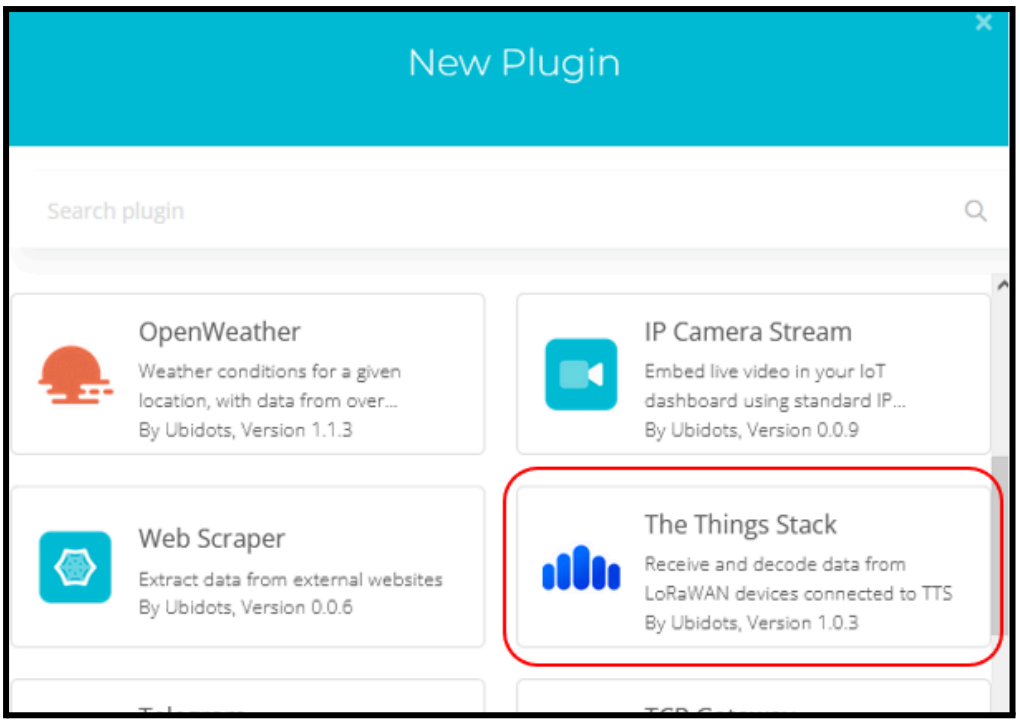
Pour faire une intégration avec Ubidots nous nous dirigeons dans TTN ⇒ Intégration⇒Webhook;
Ensuite nous cherchons Ubidots



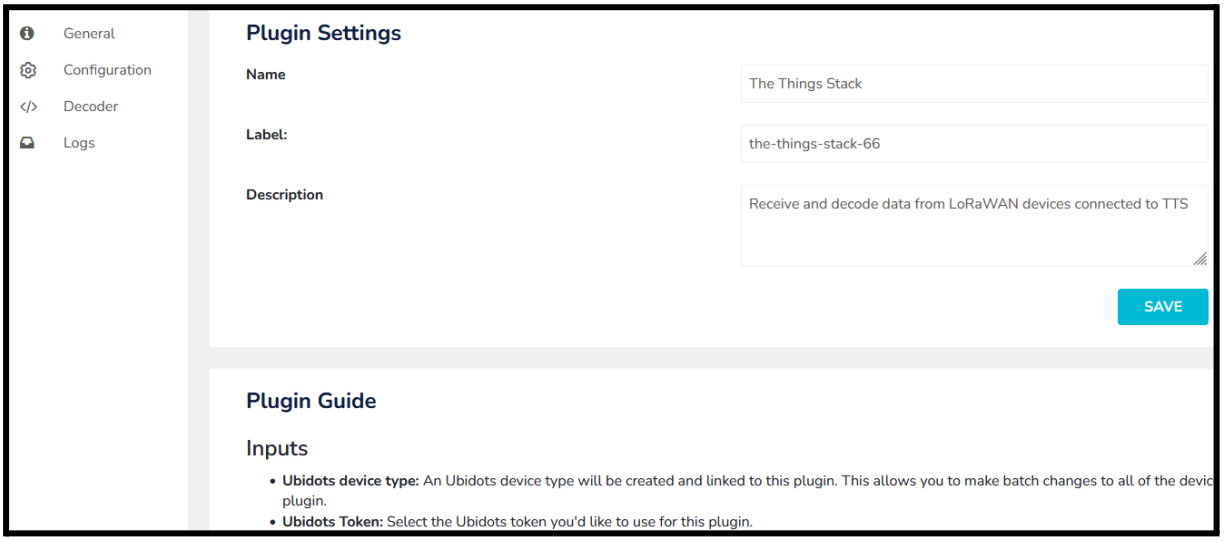
Par la suite nous devons créer un compte, Ubidots (https://industrial.ubidots.com/accounts/signin/) et créer un plugins dans le menu déroulant Devices⇒ Plugins et nous cliquons sur le bouton (+)



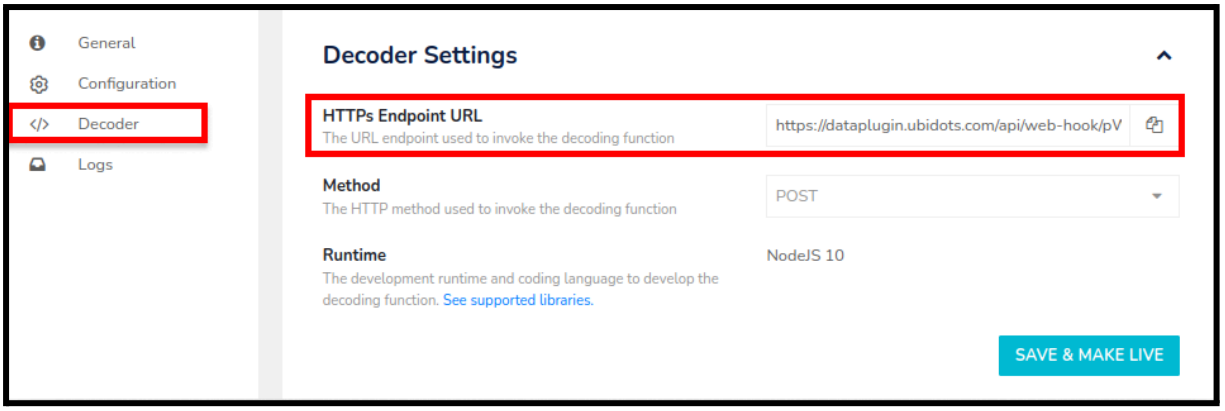
Par la suite un menu apparaîtra et dans ce menu nous devons choisir la plateforme par laquelle nous devons recevoir nos données et qui utilise la technologie LoRaWaN. Dans ubidots elle se fait par "**The Things Stack**"



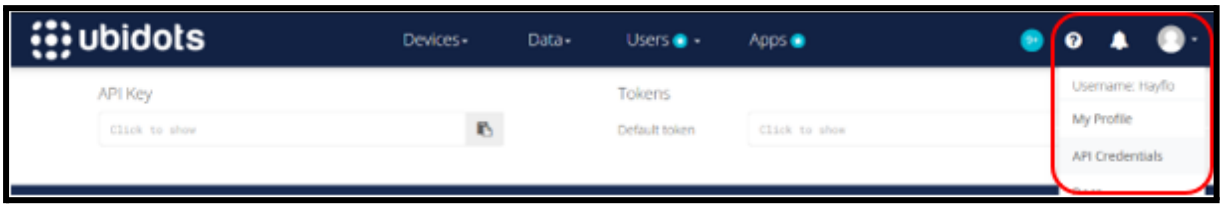
Ensuite nous nommons notre plugin. Juste en dessous dans **General** en dessous dans **Plugin Guide** nous retrouvons le guide pour connecter Ubidots à TTN



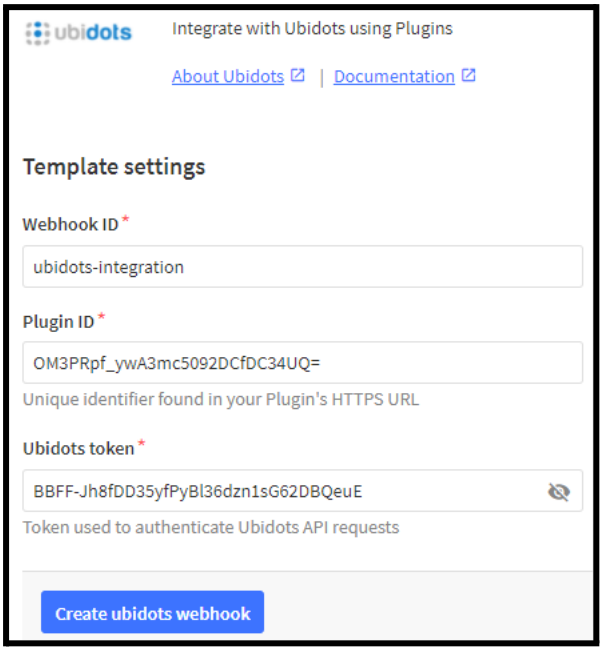
Dans décoder nous retrouvons le "**Plugin ID**" qui nous servira à faire l'intégration sur TTN.
Qu'est ce que c'est ? C'est une partie de notre URL d'extrémité HTTPS "/api/web-hook/". Voir l'exemple ci dessous



Après avoir copié le Plugin ID, nous devons copier, nous devons copier le "**tokens**" présent sur notre profil dans **API Credicals.**

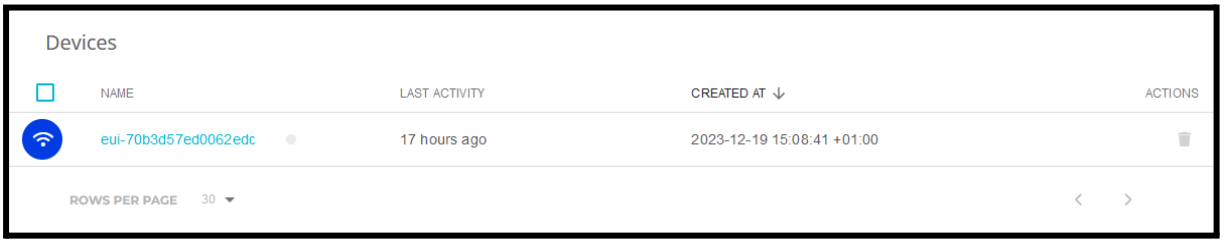


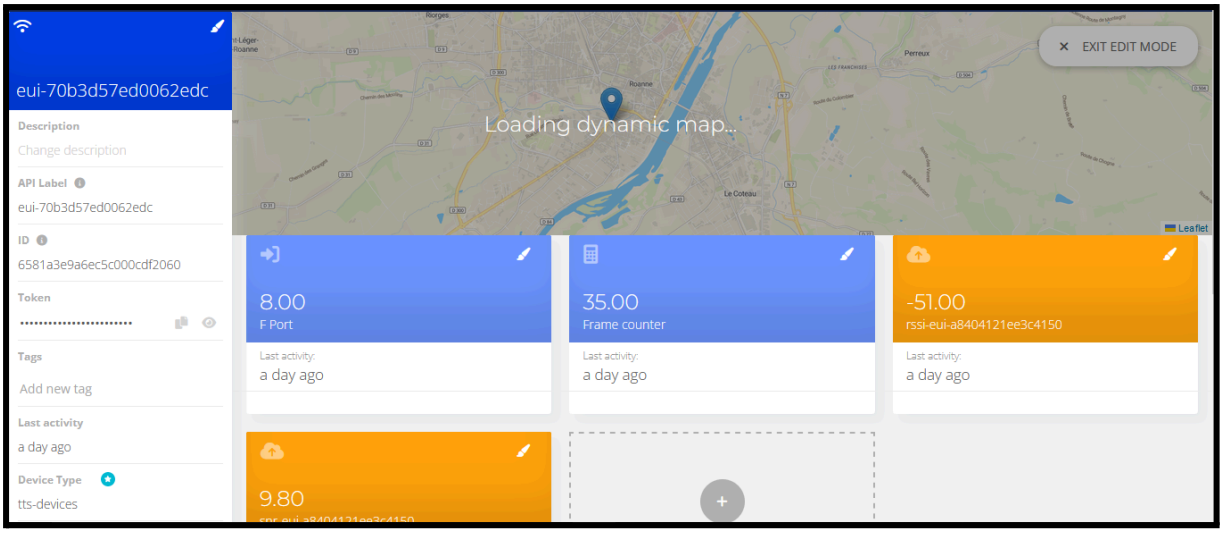
Après avoir copié ces deux éléments nous retournons sur TTN pour finaliser l'intégration. Et nous copions nos deux éléments.



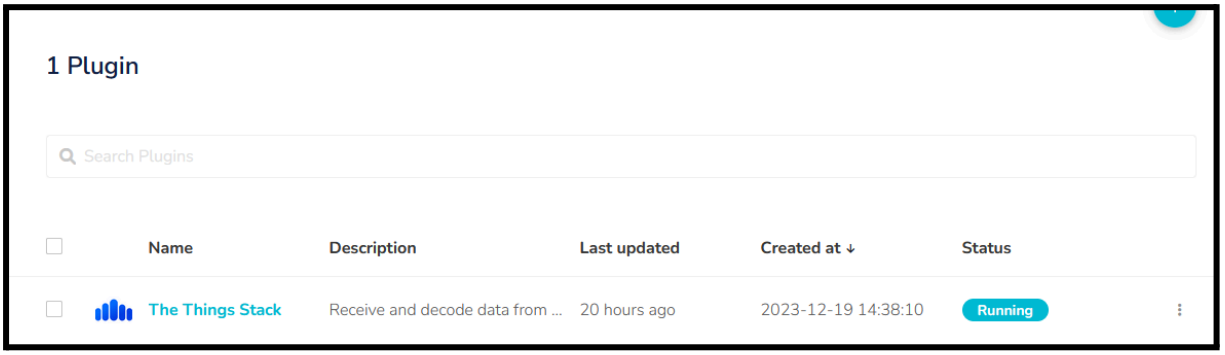
Au final lorsque l'intégration est bien faite dans devices nous devons pouvoir visualiser notre application TTN si ce n'est pas le cas cela voudrait dire que l'intégration n'a pas fonctionné.

Si nous revenons sur ubidots dans devices nous verrons l'ID de notre application et si nous cliquons dessus nous verrons normalement nos variables variables transmises de TTN





Et dans plugin nous devons avoir notre service actif.



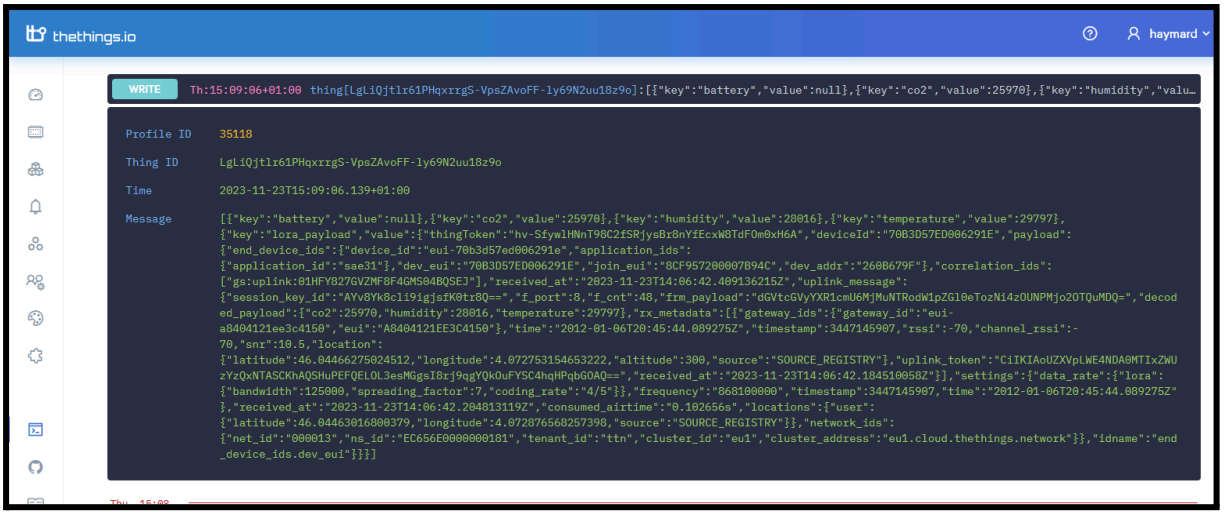
toujours dans plugins nous avons le programme javascript de base déjà fourni qui sert à décoder les données venant de TTN.

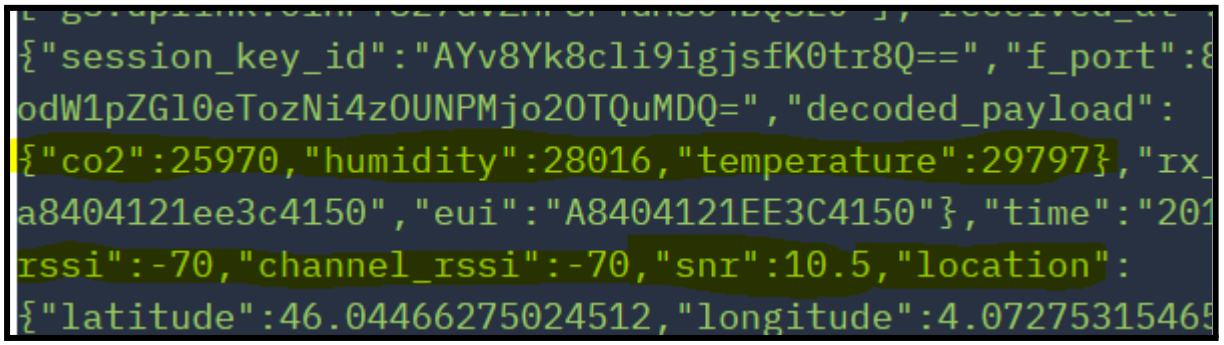
```
1   async function formatPayload(args){
2     var ubidots_payload = {};
3     // Log received data for debugging purposes:
4     // console.log(JSON.stringify(args));
5     // Get RSSI and SNR variables using gateways data:
6     var gateways = args['uplink_message']['rx_metadata'];
7     for (const i in gateways) {
8       // Get gateway EUI and name
9       var gw = gateways[i];
10      var gw_eui = gw['gateway_ids']['eui'];
11      var gw_id = gw['gateway_ids']['gateway_id'];
12      // Build RSSI and SNR variables
13      ubidots_payload['rssi-' + gw_id] = {
14        "value": gw['rssi'],
15        "context": {
16          "channel_index": gw['channel_index'],
17          "channel_rssi": gw['channel_rssi'],
18          "gw_eui": gw_eui,
19          "gw_id": gw_id,
20          "uplink_token": gw['uplink_token']
21        }
22      }
23      ubidots_payload['snr-' + gw_id] = gw['snr'];
24
25    }
26
27    // Get Fcnt and Port variables:
28    ubidots_payload['f_cnt'] = args['uplink_message']['f_cnt'];
29    ubidots_payload['f_port'] = args['uplink_message']['f_port'];
```

Mais pour pouvoir récupérer les données correctement nous devons avoir une bonne connaissance du JavaScript ce qui n'était pas notre cas donc nous n'avons pu récupérer que le SNR, le RSSI mais nous n'avons pas pu faire un programme fonctionnel qui puissent nous extraire CO2, la température, l'humidité.

WEBHOOK (Thethings.io)

Après avoir suivi le guide d'intégration de The things.io, présent dans nous avons pu récupérer les informations de notre charge utile (payload), en vérifiant dans les log, mais pour pouvoir aller plus loin et visualiser nos data sur un graphique, il est nécessaire de prendre un abonnement.





Dashboard - MQTT (grafana)